

4) SPACE complexity.

→ BFS

* Needs a queue that can hold up to $O(N)$ nodes in the worst case.

* Also requires adjacency list storage: $O(N+E)$

* Space: $O(N+E)$

→ DFS

* Needs recursion stack (or explicit stack)

* In worst case, recursion depth = $O(N)$

* Plus adjacency list: $O(N+E)$

* Space: $O(N+E)$

5) sparse vs Dense Graphs.

→ sparse graph: $E \approx O(N)$

* complexity: $O(N+E) \approx O(N)$

→ Dense Graph: $E \approx O(N^2)$

* complexity: $O(N+E) \approx O(N^2)$

6) Assumptions:-

→ Graph stored as adjacency list (not adjacency matrix, since that would be $O(N^2)$ space)

→ Graph is connected; otherwise BFS/DFS may be such on each component.

BFS & DFS (Time & Space complexity)

(1) Algorithm explanation with graph

* BFS (Breadth first search)

- * Express level by starting from the source
- * Visit all neighbours of the current node before moving deeps

* Guarantees shortest path in an unweight graph

→ DFS (Depth First search)

- * Explores as deep as possible along one branch before back tracking

* Uses recursion (or stack) to track & explain paths

* Does not guarantee short path.

(2) Data structures used

→ BFS :- Queue (FIFO)

→ DFS :- Stack (LIFO) or recursion.

3> Time complexity

→ Both BFS and DFS must visit all vertices

(N) and traverse all edges (E)

→ Each vertex is enqueued / pushed once and dequeued / popped once.

→ Each edge is checked at most twice (undirected graph)

Time complexity : $O(N+E)$